Discrete Dynamics （2）

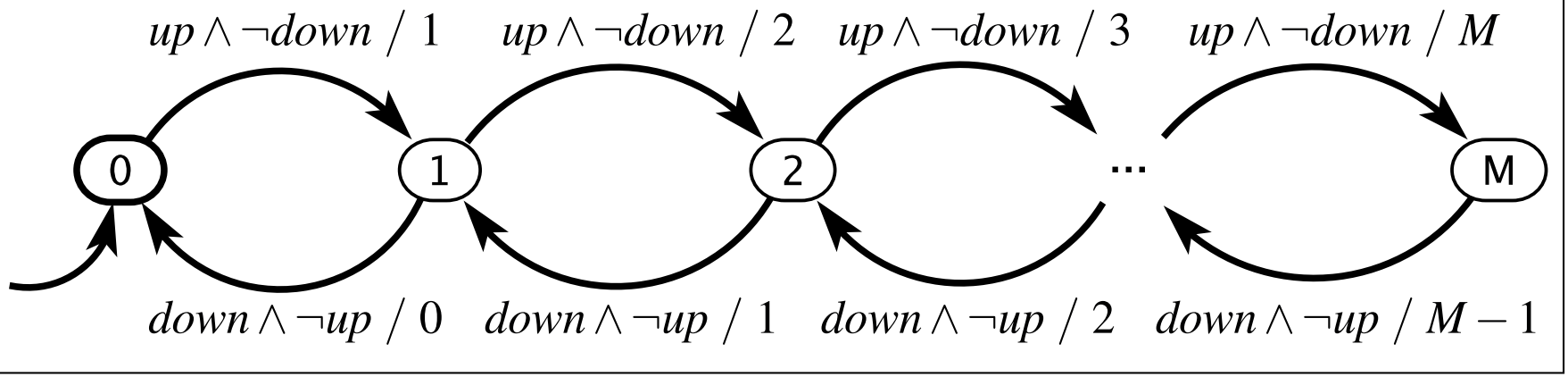
Modeling Modal Behavior

Extended State Machines

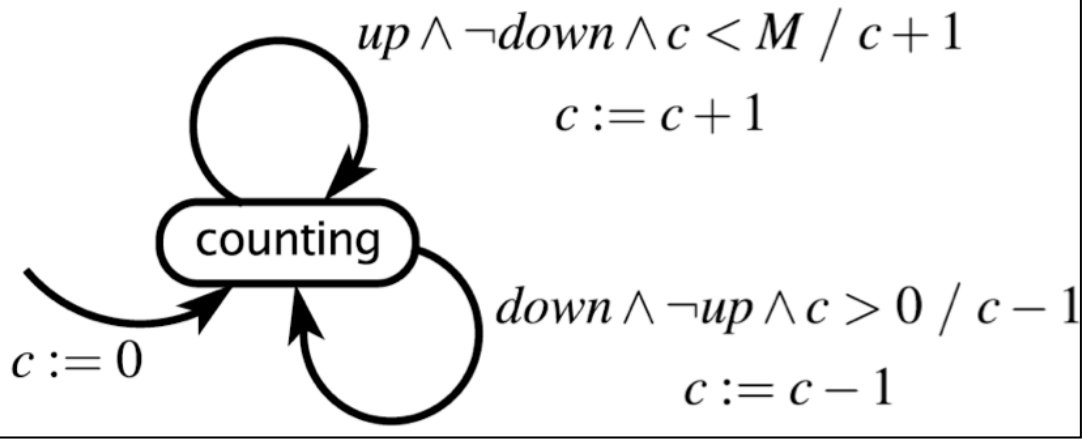
- Thinking garage, If *M* is large, the bubble-and-arc notation becomes uncontrollably.
- An **extended state machine** solves this problem by augmenting the FSM model with variables .
- Garage example

Extended State Machines

inputs: *up*, *down* : pure
output: *count* : $\{0, \cdots, M\}$



variable: c: $\{0, \cdots, M\}$
inputs: *up*, *down*: pure
output: *count*: $\{0, \cdots, M\}$



Extended State Machines

- difference from the basic FSM notation
  - ✦ variable declarations are shown explicitly
  - ✦ variables may be initialized in the initial state
  - ✦ transition annotations now have the form

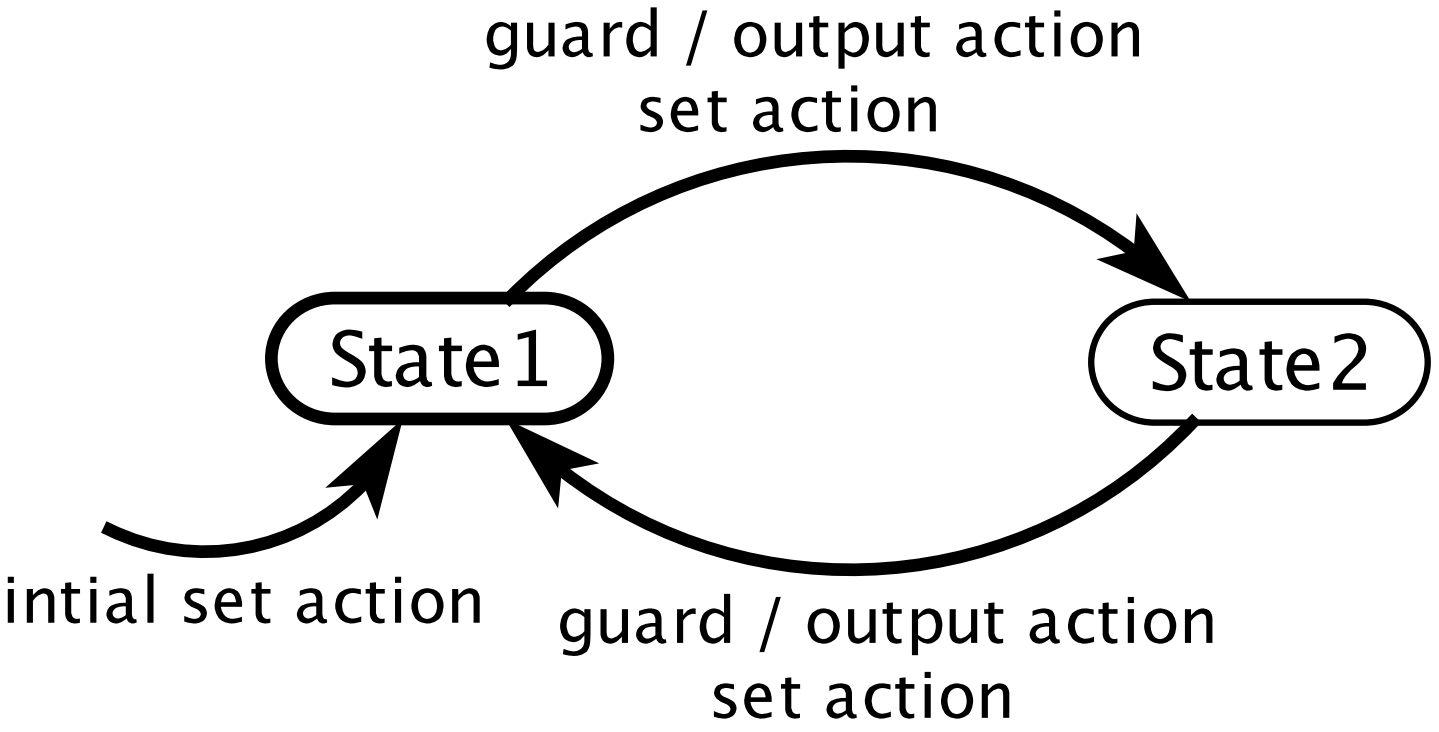
guard / output action
set action

Extended State Machines

- Set action: assignments to variables
  - ✦ These assignments are made after the guard has been evaluated and the outputs have been produced

Extended State Machines in picture

variable declaration(s)
input declaration(s)
output declaration(s)



- Question:

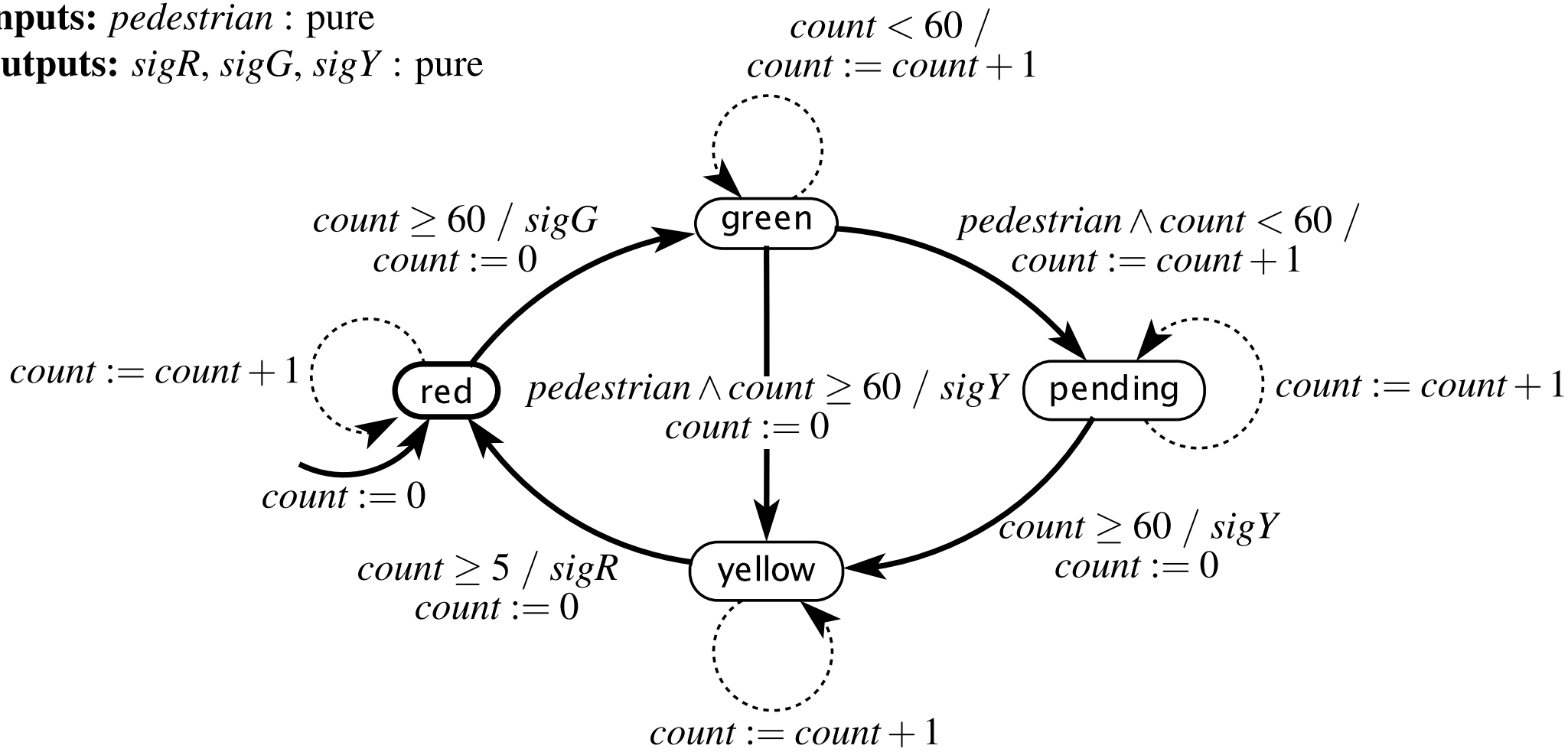
  Moore and Mealy Machines??

Example: Traffic Light Controller

- describing a traffic light at a pedestrian crosswalk
- time triggered machine that assumes it will react once per second.
- **green** for at least 60 seconds
- remain **yellow** for 5 seconds before transitioning back to **red**.
- **red** for 60 second

Example: Traffic Light Controller

variable: *count*: $\{0, \cdots, 60\}$
inputs: *pedestrian* : pure
outputs: *sigR*, *sigG*, *sigY* : pure



The number of states in ESM

- If there are *n* discrete states and *m* variables each of which can have one of *p* possible values,

the number of states?

The number of states in ESM

- If there are n discrete states and m variables each of which can have one of p possible values,

$$|\mathit{States}| = np^m$$

The number of states in ESM

- example: the number of garage counter

$$M+1$$

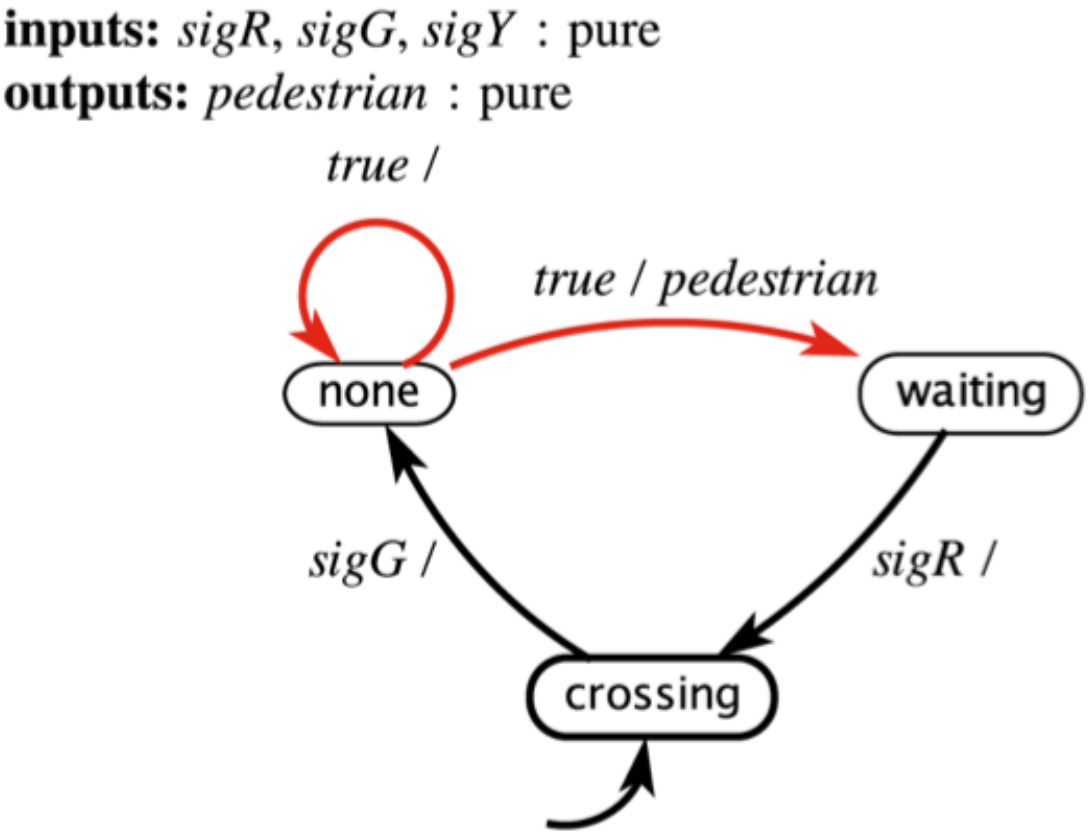
reachable states: can be reached from the initial state on some input sequence

example: the number of traffic light

the total number of states $61 \times 4 = 244$.

the number of reachable states: $61 \times 3 + 6 = 189$

Example: Nondeterminate FSM

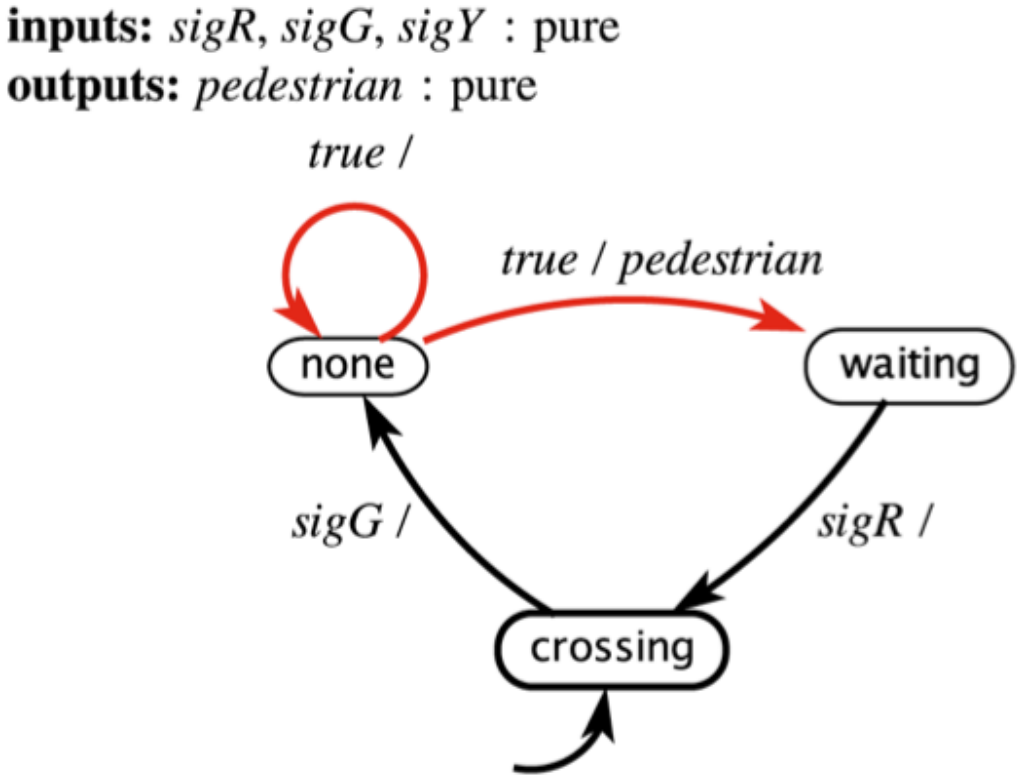


- Nondeterminate model of pedestrians arriving at a crosswalk:

- two distinct transitions with guards that can evaluate to *true* in the same reaction

- model the environment

Example: Nondeterminate FSM



- Formally, the possible update function is replaced by a function

$$possibleUpdates : States \times Inputs \rightarrow 2^{States \times Outputs}$$

- *initialStates* is a set of initial states.

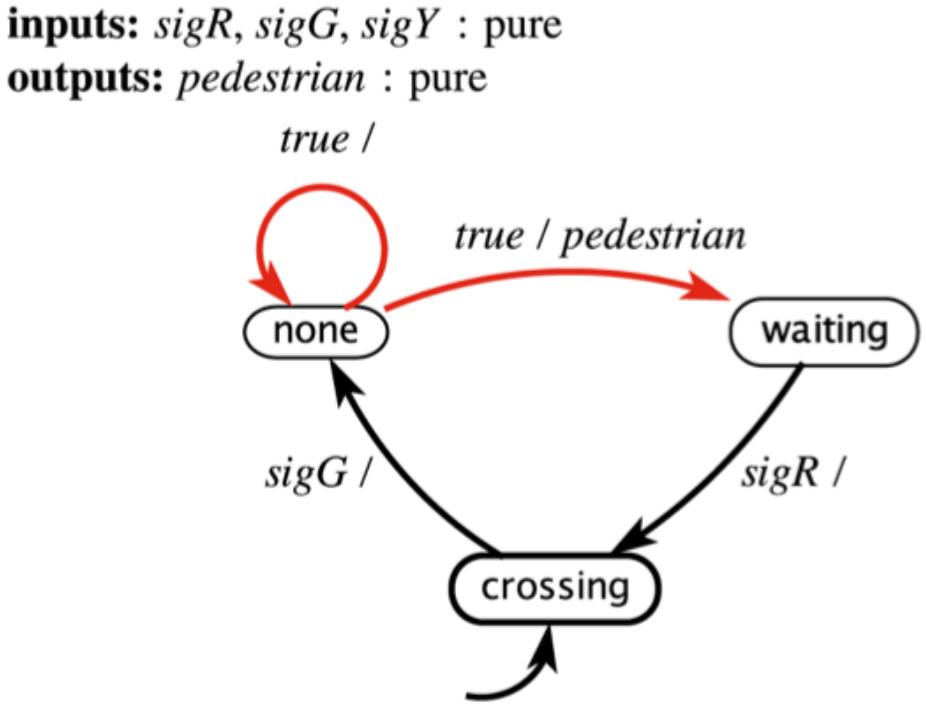
Formal model for ND FSM

($States$, $Inputs$, $Outputs$, $possibleUpdates$, $initialStates$)

- $States$ is a finite set of states;
- $Inputs$ is a set of input valuations;
- $Outputs$ is a set of output valuations;
- $possibleUpdates$:

  $States \times Inputs \rightarrow 2^{States \times Outputs}$;
- $initialStates$ is a set of initial states.

Example: Nondeterminate FSM



- formally
- (*States*, *Inputs*, *Outputs*, *possibleUpdates*, *initialStates*)

States? *Inputs?* *Outputs?*

initialStates? *possibleUpdates?*

$$
\begin{aligned}
States &= \{\text{none}, \text{waiting}, \text{crossing}\} \\
Inputs &= (\{sigG, sigY, sigR\} \rightarrow \{present, absent\}) \\
Outputs &= (\{pedestrian\} \rightarrow \{present, absent\}) \\
initialStates &= \{\text{crossing}\}
\end{aligned}
$$

The update relation is given below:

$$
possibleUpdates(s,i) = \begin{cases}
\{(\text{none}, absent)\} \\
\qquad \text{if } s = \text{crossing} \\
\qquad \wedge\, i(sigG) = present \\
\{(\text{none}, absent), (\text{waiting}, present)\} \\
\qquad \text{if } s = \text{none} \\
\{(\text{crossing}, absent)\} \\
\qquad \text{if } s = \text{waiting} \\
\qquad \wedge\, i(sigR) = present \\
\{(s, absent)\} \qquad \text{otherwise}
\end{cases}
$$

Uses of nondeterminism

- Modeling unknown aspects of the environment or system

Such as: how the environment changes the iRobot's orientation

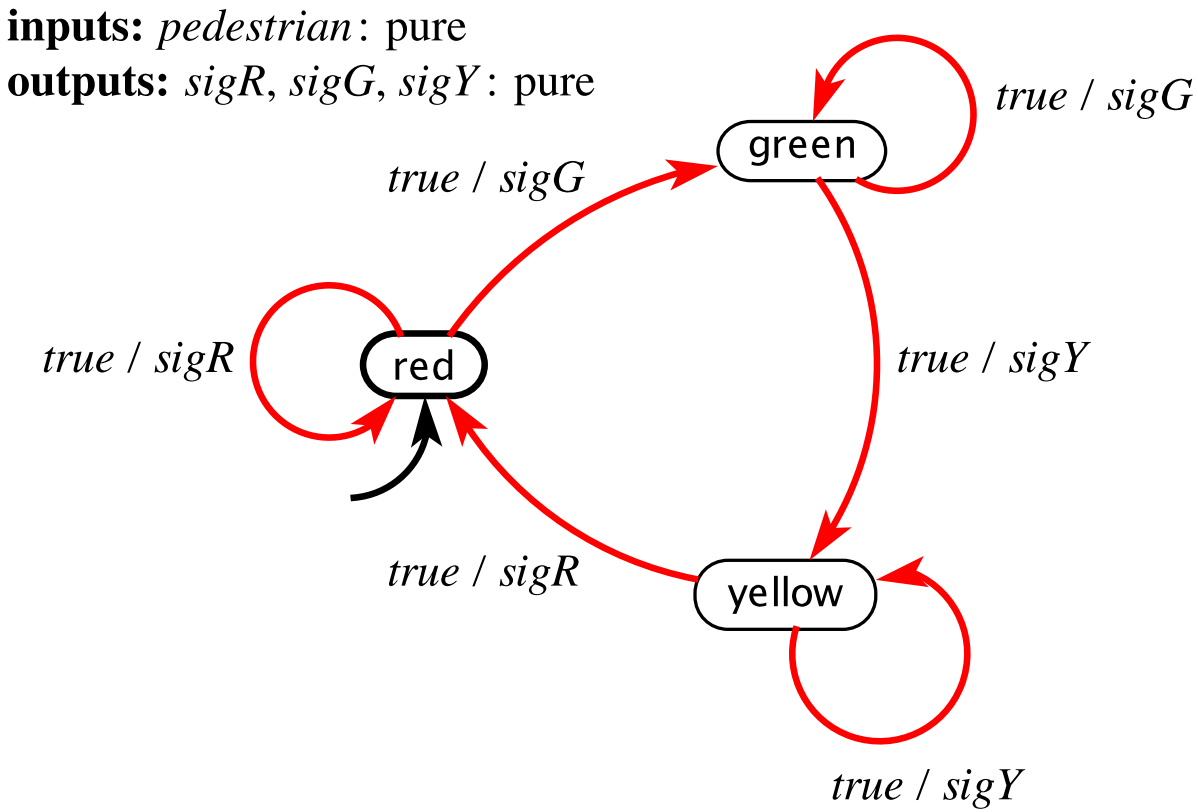
- Hiding detail in a *specification* of the system

We will see an example of this later (see notes)

Any other reasons why nondeterministic FSMs might be preferred over deterministic FSMs?

specification

- consider a specification that the traffic light cycles through red, green, yellow,

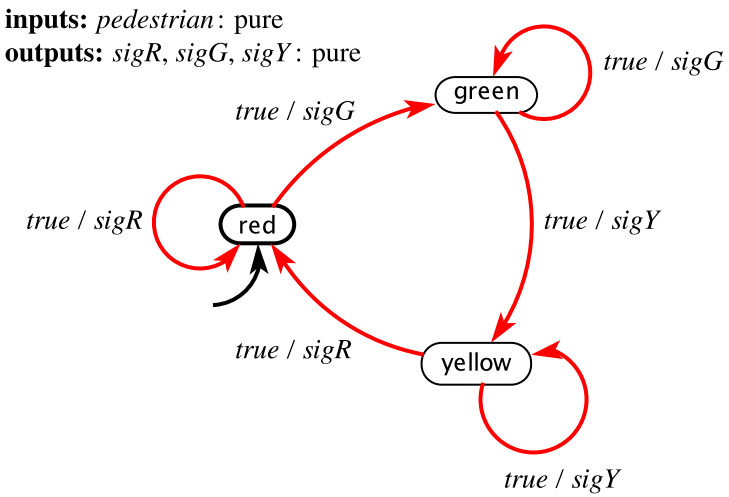


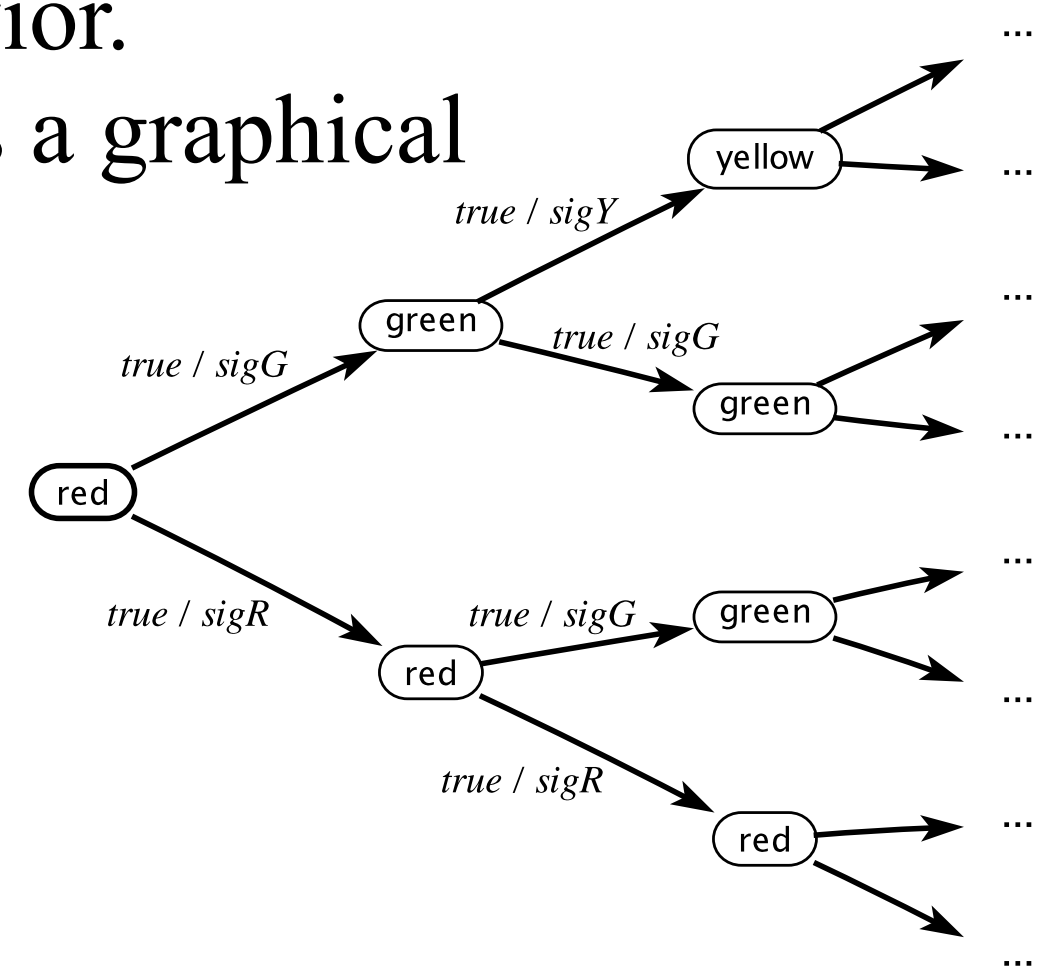
Size Matters

- Non-deterministic FSMs are more compact than deterministic FSMs
- ND FSM→D FSM: Exponential blow-up in states in worst case
- How to translate from ND FSM to D FSM???

Behaviors and Traces

- FSM **behavior** is a sequence of (non-stuttering) steps.
- A **trace** is the record of inputs, states, and outputs in a behavior.
- A **computation tree** is a graphical representation of all possible traces.





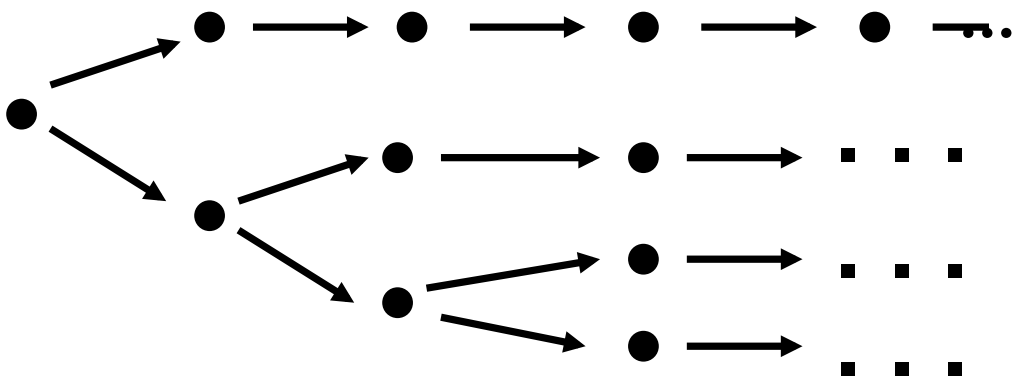
Non-deterministic Behavior: Tree of Computations

For a fixed input sequence:

- A deterministic system exhibits a single behavior
- A non-deterministic system exhibits a **set of behaviors**

D FSM behavior for a particular input sequence:

ND FSM behavior for an input sequence:



Behaviors and Traces

- FSMs are suitable for formal analysis.
- For example,
  - **safety** analysis might show that some unsafe state is not reachable.

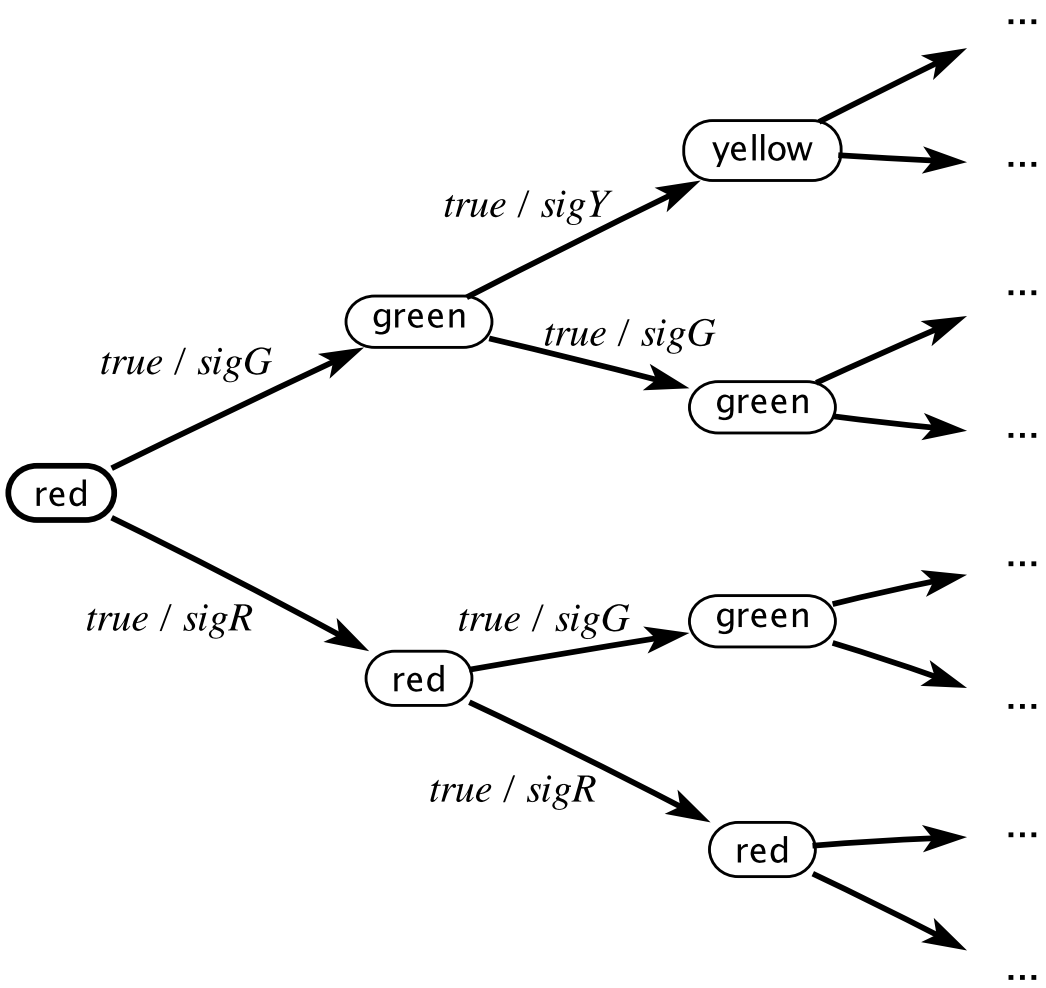
Examples of Behaviors

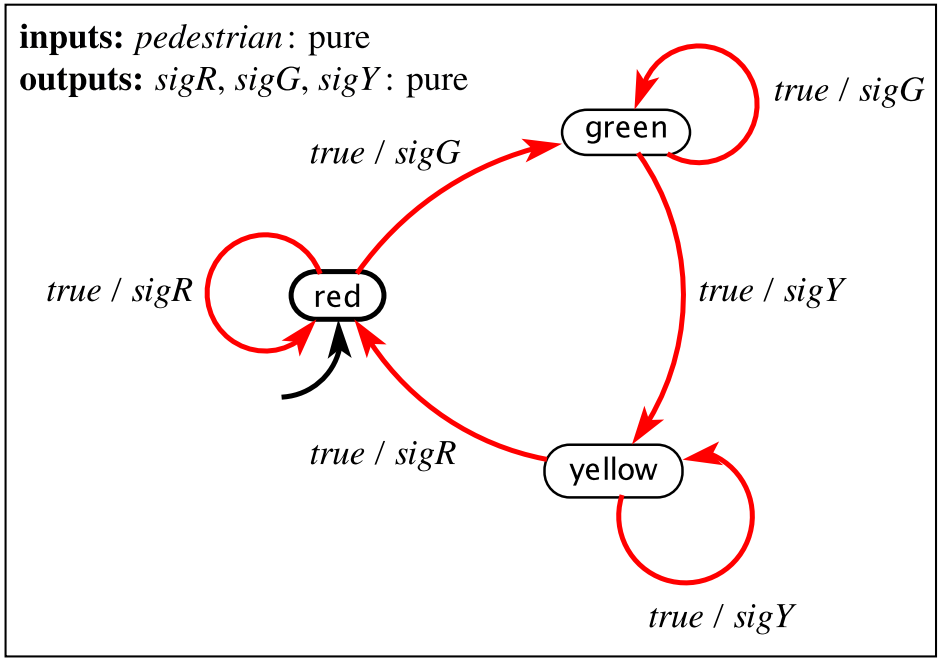
- The set of all behaviors of a state machine *M* is called its language, written *L*(*M*).
- Garage counter

$$0 \xrightarrow{up \wedge down \ /} 0 \xrightarrow{\ /\ } 0 \xrightarrow{up \ / \ 1} 1 \xrightarrow{down \ / \ 0} 0 \xrightarrow{up \ / \ 1} \cdots$$

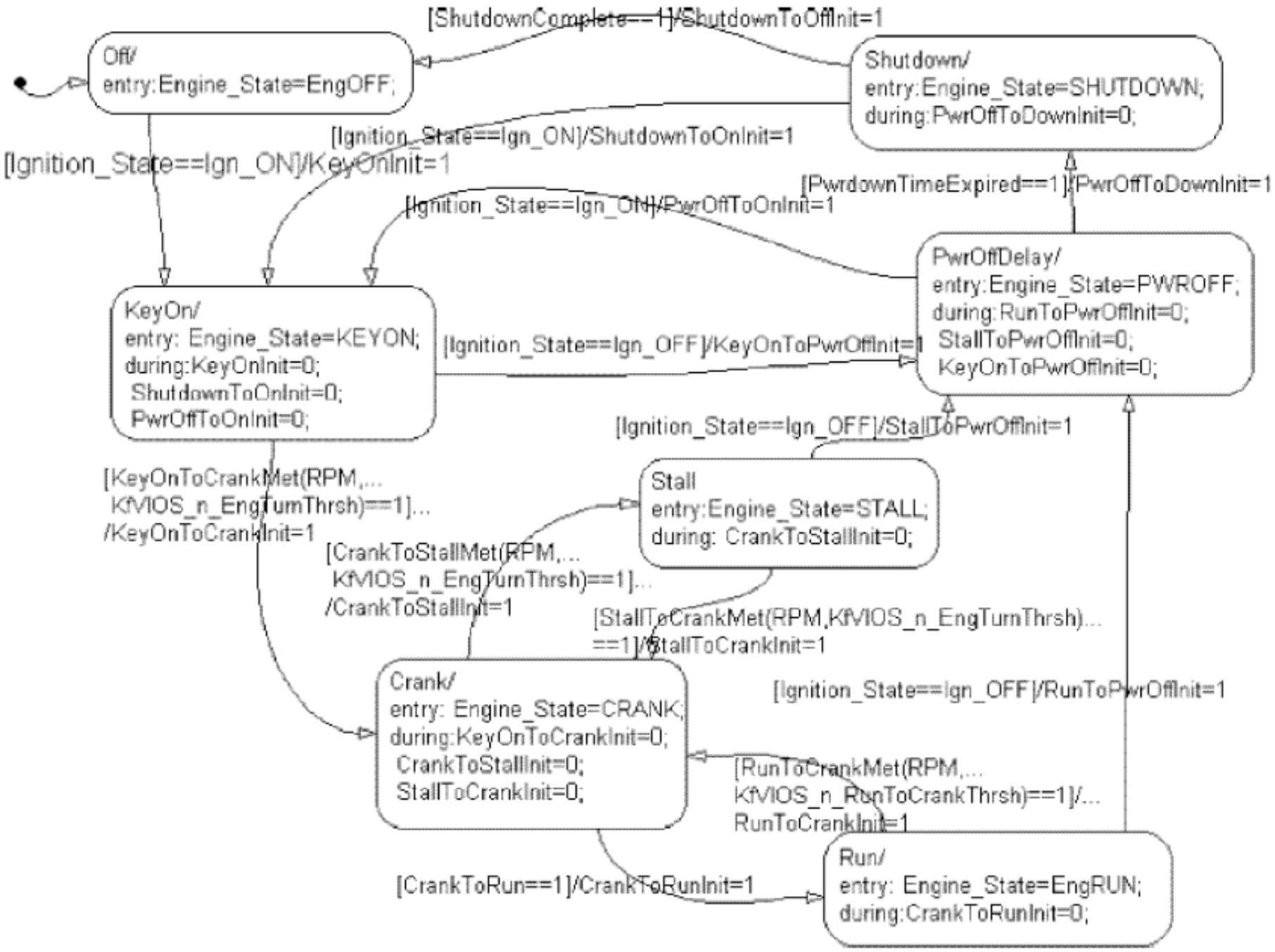
Examples of Traces

- specification's traces of traffic light





Example from Industry: Engine Control



Elements of a Modal Model (FSM)

What we will be able to do with FSMs

FSMs provide:

1. A way to represent the system for:
   - Mathematical analysis
   - So that a computer program can manipulate it
2. A way to model the environment of a system.
3. A way to represent what the system *must* do and *must* not do – its specification.
4. A way to check whether the system satisfies its specification in its operating environment.

Representing a state machine

1. Pictorial notation

2. Table representing transition relation

3. Functional notation

When would you use each representation?

summary

- state machines to model systems with discrete dynamics
- a graphical notation for finite state machines
- an extended state machine notation
- a mathematical model
- The mathematical model to prove properties of a model.